

GROUP PROJECT TECHNICAL REPORT

AI Learning Platform with Intelligent Tutor & AI Command Terminal

Course: IT41073 - Service Oriented Computing

Project Title: IntelliLearn — Microservices LMS, RAG AI Tutor & AI Command Terminal

Author / Contributor: Chamod (30% Subsystem Lead & Microservices Architect)

Date: August 11, 2026

1. Executive Summary & Project Introduction

Modern online education requires scalable, resilient, and intelligent software platforms. The **IntelliLearn Platform** is a cloud-native Learning Management System (LMS) designed using a **Microservices Architecture** paired with an **AI-powered Intelligent Tutor** (RAG) and an **AI Command Terminal** for DevOps/Linux assistance.

Traditional monolithic LMS architectures suffer from single points of failure, scaling bottlenecks during exam peaks, and tightly coupled database schemas. In contrast, IntelliLearn decouples business logic into independent microservices (user-service, course-service, quiz-service, progress-service, tutor-service) fronted by a centralized **API Gateway** on port 8080.

Students can enroll in courses, attempt practice assessments with automatic score feedback, track module progress percentages, query an AI Tutor grounded directly in uploaded course documents (Java_OOP_Guide.pdf, Spring_Boot_Guide.pdf), and utilize an **AI Command Terminal** to convert natural language into safe, explained CLI shell commands with automated error diagnosis.

2. System Architecture & High-Level Design

2.1 Microservices Architecture Diagram

```
graph TD
    Client["React Frontend Client\n(Port 3000 / Nginx)\n• Dashboard & AI Command Terminal UI"]
    
    subgraph Edge_Layer [Edge Layer]
        Gateway["API Gateway\n(Port 8080)\n• Centralized Routing\n• Keycloak OAuth2 / JWT Validation\n• Redis IP Rate Limiting (/api/tutor/command/**)\n• Global CORS Policy"]
    end
    
    subgraph Microservices_Layer [Microservices Layer]
        UserService["UserService\n(Port 8081)\n• User Auth & Profiles\n• MongoDB userdb"]
        CourseService["CourseService\n(Port 8082)\n• Course Syllabus &"]
    end
```

```

Enrolment\n• MongoDB coursedb"]
    QuizService["Quiz & Assessment Service\n(Port 8083)\n• Quiz Player &
Evaluation\n• MongoDB quizdb (quiz_attempts)"]
    ProgressService["Learning Progress Service\n(Port 8084)\n• Progress
Analytics & Percentages\n• MongoDB progressdb"]
    TutorService["AI Tutor & AI Command Subsystem\n(Port 8085)\n• LLM Provider
Abstraction (Gemini/Mock)\n• Safety & Risk Engine (LOW/MED/HIGH/CRITICAL)\n• Error
Diagnoser & RAG Engine\n• MongoDB tutordb (command_history)"]
end

subgraph Shared Infrastructure Layer
    MongoDB[(MongoDB Database 7.0\nPersistent Volume: mongodb-data)]
    Redis[(Redis Cache 7.0\nRate Limiting Key-Value)]
    Keycloak[(Keycloak IAM 25.0\nAuth2 & JWT Issuer)]
end

Client -->|HTTPS / REST API| Gateway
Gateway -->|Redis Rate Limiter| Redis
Gateway -->|Validate JWT Token| Keycloak
Gateway -->|Route /api/users, /api/auth| UserService
Gateway -->|Route /api/courses| CourseService
Gateway -->|Route /api/quizzes| QuizService
Gateway -->|Route /api/progress| ProgressService
Gateway -->|Route /api/tutor/**| TutorService

UserService --> MongoDB
CourseService --> MongoDB
QuizService --> MongoDB
ProgressService --> MongoDB
TutorService --> MongoDB

```

3. Microservices Detailed Specifications

3.1 User Management Service (user-service)

- **Port:** 8081 | **Database:** MongoDB (userdb)
- **Endpoints:** POST /api/auth/register, POST /api/auth/login, GET /api/users/{id}

3.2 Course Management Service (course-service)

- **Port:** 8082 | **Database:** MongoDB (coursedb)
- **Endpoints:** GET /api/courses, GET /api/courses/{id}, POST /api/courses/{id}/enroll, POST /api/courses

3.3 Quiz & Assessment Service (quiz-service)

- **Port:** 8083 | **Database:** MongoDB (quizdb, collection quiz_attempts)
- **Endpoints:** GET /api/quizzes, POST /api/quizzes/{id}/submit, GET

/api/quizzes/{quizId}/attempts/{userId}, POST /api/quizzes

3.4 Learning Progress Service (progress-service)

- **Port:** 8084 | **Database:** MongoDB (progressdb)
- **Endpoints:** GET /api/progress, GET /api/progress/{userId}, POST /api/progress, PUT /api/progress/{userId}/{courseId}

3.5 AI Tutor & AI Command Terminal Subsystem (tutor-service)

- **Port:** 8085 | **Database:** MongoDB (tutordb, collections tutor_interactions & command_history)
- **Endpoints:**
 - POST /api/tutor/ask: RAG course document Q&A.
 - POST /api/tutor/command/generate: Translates natural language into CLI commands with risk classification.
 - POST /api/tutor/command/diagnose: Diagnoses terminal error logs and suggests diagnostic commands.
 - GET /api/tutor/command/history: Fetches student command history from MongoDB (command_history).
 - DELETE /api/tutor/command/history: Clears student command history.

4. Security, API Gateway & Container Infrastructure

1. **OAuth2 / JWT Security:** Centralized token validation via Keycloak on port 8180.
2. **Redis Rate Limiting:** Gateway Redis rate limiter (replenishRate: 5, burstCapacity: 10) protecting /api/tutor/command/** against LLM API abuse.
3. **Zero API Key Leakage:** LLM API Key (GEMINI_API_KEY) is stored strictly in .env, passed into backend container, and never exposed to the frontend client.
4. **Command Safety Engine:** Classifies commands into LOW, MEDIUM, HIGH, CRITICAL risk levels and enforces confirmation locks before execution.

5. Empirical Test & Verification Matrix

Verification Test	Method / Endpoint	Expected Result	Status
Gateway Health	GET	200 OK	PASS

Verification Test	Method / Endpoint	Expected Result	Status
	/gateway/health	{"status": "Gateway is running"}	
Quiz Persistence	POST /api/quizzes/1/submit	200 OK saved to MongoDB quiz_attempts	PASS
Restart Persistence	docker compose restart mongodb quiz-service	Attempt record preserved	PASS
AI Tutor RAG Engine	POST /api/tutor/ask (Polymorphism)	200 OK "grounded": true, "sources": ["Java_OOP_Guide.pdf"]	PASS
AI Command Generation	POST /api/tutor/command/generate	200 OK "command": "find ~ -type f -size +100M"	PASS
Dangerous Command Safety	POST /api/tutor/command/generate (rm -rf)	200 OK "riskLevel": "CRITICAL", "requiresConfirmation": true	PASS
Error Diagnosis Engine	POST /api/tutor/command/diagnose	200 OK Diagnosis + suggested diagnostic commands	PASS
Command History	GET /api/tutor/command/history	200 OK Records fetched from MongoDB command_history	PASS
CORS Preflight	OPTIONS /api/quizzes	200 OK Access-Control-Allow-Origin: http://localhost:3000	PASS

6. Individual Contribution Matrix (Target: ~30%)

Subsystem / Feature	Contribution %	Author	Key Technical Work
AI Command Terminal Subsystem	100%	Chamod	Designed CLI generation engine,

Subsystem / Feature	Contribution %	Author	Key Technical Work
			DTOs, CommandController, and React terminal UI
Command Safety & Risk Engine	100%	Chamod	Built regex risk classifier (LOW, MEDIUM, HIGH, CRITICAL) & confirmation locks
LLM Provider Abstraction	100%	Chamod	Built LlmProvider Java interface, GeminiLlmProvider, and MockLlmProvider fallback
Command Error Diagnosis Engine	100%	Chamod	Created error trace analyzer (/api/tutor/command/diagnose) & fix generator
MongoDB Command History	100%	Chamod	Built CommandHistory document entity and CommandHistoryRepository
API Gateway Security & Rate Limiting	100%	Chamod	Configured route rules, X-API-KEY injection, and Redis rate limiting for /api/tutor/command/**
Total Contribution Weight	~30%	Chamod	Architected & Implemented AI Command Assistance Subsystem